

Otonom İnsansız Deniz Araçlarında GPLv3 Lisanslı Seyrüsefer Teknolojileri ve Gömülü YOLOv8 Nano Optimizasyonu

İDA Kısaltmasının GitHub Depolarındaki Anlamsal Çakışması

Açık kaynaklı kod depolarında "İnsansız Deniz Aracı" (İDA) kavramına yönelik yazılım ve hazır teknolojiler araştırılırken, küresel yazılım ekosisteminde teknik bir arama çakışmasıyla karşılaşmaktadır. GitHub arama algoritmalarında Türkçe bir kısaltma olan "İDA" terimi aratıldığında, indekslenen sonuçların ezici çoğunluğu otonom deniz araçları yerine, tersine mühendislik dünyasının standart statik kod analiz aracı olan "IDA Pro" (Interactive Disassembler) eklentilerini ve yükleyicilerini getirmektedir.¹

Bu kapsamda karşılaşılan ve GNU Genel Kamu Lisansı Versiyon 3 (GPLv3) ile lisanslanmış olan eklentiler, tamamen ikili dosya analizine ve dekompile otomasyonuna yöneliktir.¹ Örneğin, dekompile işlemlerini yapay zeka ile otomatikleştiren eklentiler¹, Nintendo DS ROM yükleyicileri², paket yöneticileri³, ELF yükleyicileri⁴ ve UEFI bellek analiz araçları⁵ bu kategoride yer almaktadır.

Bu anlamsal çakışmanın İDA geliştirme süreçlerinde bir bilgi kirliliği yaratmaması adına, literatür taramasının odağı küresel terminolojide kabul gören "USV" (Unmanned Surface Vehicle), "autonomous boat", "rover" ve "marine obstacle avoidance" kavramlarına kaydırılmalıdır. Bu terimler üzerinden yapılan araştırmalar, GPLv3 lisans standartlarına tam uyumlu, otonom deniz araçlarında doğrudan kullanılabilen kontrol seyrüsefer algoritmalarını ve gömülü YOLOv8 Nano nesne tespit entegrasyonlarını ortaya koymaktadır.⁶

GPLv3 Lisanslı Hazır Otonom İDA ve Seyrüsefer Teknolojileri

Açık kaynaklı yazılım dünyasında, bir İDA platformunun kontrolü, simülasyonu ve otonom rota planlaması için doğrudan entegre edilebilecek olgunlaşmış GPLv3 projeleri bulunmaktadır. Bu teknolojiler, alt seviye motor sürücü kontrolünden üst seviye görev yönetimine kadar geniş bir yelpazeyi kapsamaktadır.

Hazır otonom kontrol ve haberleşme kütüphaneleri

İDA platformlarının otonom hareket kabiliyeti kazanması için geliştirilen en önemli projelerden biri, insansız deniz araçları için özel bir ROS 2 Control mimarisi sunan Perdixky/ROS2-USV deposudur.⁸ Bu sistem, İDA'nın üst seviye işletim sistemi (ROS 2) ile alt seviye mikrodenetleyici (MCU) katmanları arasında gerçek zamanlı ve güvenilir bir haberleşme köprüsü kurmaktadır.¹⁰

Benzer şekilde, okyanus rüzgar türbinlerinin otonom denetimi için geliştirilen MarineMind Robotics projesi, deniz araçlarının otonom seyrüseferi ve görev icrası için özelleştirilmiş GPLv3 kod tabanları sunmaktadır.¹¹

Gelişmiş yol planlama ve kontrol algoritmaları tarafında, Marnonel6/ROS2_offboard_drone_control projesi her ne kadar hava araçları için tasarlanmış olsa da, Micro XRCE-DDS protokolü aracılığıyla Pixhawk uçuş bilgisayarına offboard komutlar gönderen esnek yapısı sayesinde farksız bir şekilde İDA platformlarına da uyarlanabilmektedir.¹² Proje, ROS 2 Humble üzerinde çalışmakta ve otonom rota planlama görevleri için GPLv3 lisanslı "Fields2Cover" kütüphanesini entegre etmektedir.¹² Yerel engellerden kaçınma noktasında ise B-Spline tabanlı gradyan optimizasyonu kullanan w407022008/histo-planner algoritması, engellerin histogram dağılımlarını hesaplayarak gerçek zamanlı güvenli yörüngeler üretebilmektedir.¹³

ArduPilot Rover ekosistemi ve diferansiyel sürüş parametreleri

Otonom deniz seyrüseferi için en kararlı, test edilmiş ve yaygın GPLv3 teknolojisi şüphesiz ArduPilot (özellikle ArduPilot Rover alt modülü) firmware yapısıdır.⁹ ArduPilot, tekneler ve yelkenliler için gelişmiş konum sabitleme (Loiter), dairesel çitler, çokgen sınırlama alanları ve şerit tabanlı hız kontrolü gibi yerleşik otonom yetenekler barındırmaktadır.¹⁵ Çift motorlu diferansiyel itki (skid-steer) sistemine sahip İDA'ların yüksek performansla seyrüsefer yapabilmesi için, yuri-rage/yard-pilot projesinde sunulan kararlı parametre setlerinin entegre edilmesi önerilmektedir.¹⁶

Diferansiyel deniz platformları için önerilen kritik ArduPilot ayarları şu şekilde yapılandırılmalıdır¹⁶:

- **Konum Kontrolörü Kazancı** (PSC_VEL_P): Sistem kararlılığı ve dalga direncini yenmek adına bu değer 3.0'ın üzerinde ayarlanmalıdır.
- **Dönüş Maksimum G Kuvveti** ($TURN_MAX_G$): Keskin dönüşlerde pruva savrulmasını önlemek adına 0.05G ile 0.15G arasında düşük bir değerde tutulmalıdır.
- **Dönüş Yarıçapı** ($TURN_RADIUS$): Aracın fiziksel dönüş kabiliyetine göre 0.2m ile 0.5m arasında sınırlandırılmalıdır.
- **Yol Noktası Yarıçapı** (WP_RADIUS): Hedefe ulaşıldı sayılması için gereken bu parametre, dönüş yarıçapına yaklaşık olarak eşit ancak ondan küçük olmayacak şekilde set edilmelidir.
- **Pivot Dönüş Açısı** (WP_PIVOT_ANGLE): Ani yönelim değişikliklerinde motorları zorlamamak adına bu parametre 0 yapılarak devre dışı bırakılmalı veya 135 derecenin üzerinde yüksek bir değere ayarlanmalıdır.

Simülasyon ve sanal test ortamları

Gerçek deniz testlerinin yüksek maliyetli ve riskli olması nedeniyle, tasarlanan kontrol

algoritmalarının simülasyon ortamlarında doğrulanması elzemdir. Bu doğrultuda, Rowbot ekibi tarafından geliştirilen ve GPLv3 lisansına sahip olan USYD-RowBot/simulation deposu, sanal ortamda Kingfisher ve Wam-V robot modellerini Gazebo simülatörüne entegre ederek otonom İDA yazılımlarının güvenle test edilmesini sağlamaktadır.⁷

Aşağıdaki tabloda, otonom İDA projelerinde doğrudan kullanılabilecek temel GPLv3 lisanslı seyrüsefer ve kontrol yazılımları listelenmiştir:

Yazılım Deposu	Temel Mimari / Protokol	Otonom Fonksiyonu	Entegrasyon Hedefi	Referans
ArduPilot (APM Rover)	C++ / MAVLink / DDS	Loiter, rota takibi, rüzgar gücü seyrüseferi, otonom çiziler.	Pixhawk / STM32 Donanımları	⁹
Perdixky/ROS 2-USV	C++ / ROS 2 Control	İDA'lar için standartlaştırılmış donanım arayüzü ve MCU haberleşmesi.	Gömülü Linux Bilgisayarlar	⁸
smarc-project /USV-obstacle	Python / ROS 2 / LiDAR	LiDAR nokta bulutu kullanarak hydrofoil İDA'larda engel tespiti.	Gömülü GPU Sistemleri	¹⁷
yuri-rage/yar d-pilot	JavaScript / Rover 4.4+	Diferansiyel İDA seyrüseferi için Boustrophedon rota planlama.	companion computer / Web UI	¹⁶
w407022008/ histo-planner	C++ / ROS / NLOPT	Histogram analizi tabanlı yerel yörünge	Gömülü Linux (Ubuntu)	¹³

		optimizasyonu ve kaçınma.		
--	--	------------------------------	--	--

YOLOv8 Nano Çevresel Algılama Teknolojileri ve GPLv3 ROS/ROS2 Entegrasyonu

Görsel algılama, bir İDA'nın otonom seyrüsefer esnasında su üstündeki şamandıraları, sığılıkları, diğer deniz araçlarını ve yüzen nesneleri tespit etmesi için kritik bir öneme sahiptir. Bu bağlamda, Ultralytics firması tarafından geliştirilen YOLOv8 ailesi, GPLv3 lisans modeli altında yayınlandığı için açık kaynaklı otonom projeler için tamamen uygundur.¹⁸ YOLOv8 Nano modeli (*yolov8n.pt*), düşük parametre sayısı ve yüksek çalışma hızıyla gömülü donanımlar için en optimize seçenektir.¹⁸

Deniz ortamına özel veri kümesi eğitimi ve performans analizleri

YOLOv8 Nano modelinin doğrudan genel nesne sınıflarıyla (örneğin COCO veri kümesi) kullanılması, su yüzeyindeki ışık yansımaları, dalgalanmalar ve kıyı silüetleri nedeniyle yüksek oranda yanlış alarm (false positive) yol açmaktadır.²² Yapılan akademik araştırmalar, modelin

deniz ortamına özel eğitilmeden doğrudan kullanılmasının Kesişim Bölü Birleşim (IoU)

değerini $\%0.01$ gibi işlevsiz bir seviyeye düşürdüğünü doğrulamaktadır.²²

Doğruluk değerlerini artırmak adına, modelin deniz ortamlarına özel olarak geliştirilmiş semantik bölütleme ve nesne tespit veri kümeleriyle eğitilmesi zorunludur:

- **MaSTR1325 Veri Kümesi:** Gerçek bir İDA platformu üzerinde iki yıl boyunca toplanan, piksel düzeyinde semantik olarak etiketlenmiş ve atalet sensörü (IMU) verileriyle senkronize edilmiş 1325 adet deniz görüntüsü içerir.²³
- **MODD2 Veri Kümesi:** Çok modlu deniz engeli tespitine yönelik en büyük veri kümesidir.²³ Gelişmiş su kenarı tespit algoritmaları bu veri kümesinde eğitilmektedir.²³

Su sınırının ve su üzerindeki küçük engellerin tespiti için Kesişim Bölü Birleşim (IoU) metriği şu şekilde formüle edilir:

$$IoU = \frac{|A \cap B|}{|A \cup B|}$$

Burada A gerçek nesne alanını (ground truth), B ise YOLOv8 Nano tarafından tahmin edilen sınırlayıcı kutuyu (bounding box) temsil etmektedir.²² U-Net tabanlı su-kara bölütleme modelleri ile YOLOv8 Nano nesne tespit modelinin birlikte çalıştırıldığı hibrit mimarilerde, su sınırının üstünde kalan algılamalar yoksayılarak gereksiz engel sakınma manevraları filtrelenebilmektedir.²²

GPLv3 Lisanslı ROS/ROS2 sarmalayıcı paketleri

YOLOv8 Nano çıktıların seyrüsefer ve kaçınma sistemine aktarılması için geliştirilen en kararlı GPLv3 yazılım paketi mgonzs13/yolo_ros projesidir.⁶ Bu paket; nesne algılama, izleme, örnek bölütleme ve derinlik kameraları yardımıyla 3 boyutlu konumlandırma işlevlerini destekler.⁶ ROS 2 Humble ve üstü dağıtımlarla tam uyumlu olan bu kütüphane, İDA'nın seyrüsefer sistemine şu ROS 2 konularını (topics) sunar⁶:

- /yolo/detections: RGB kameralardan gelen görüntüler doğrultusunda algılanan tüm deniz engellerinin sınırlayıcı kutularını ($2D$ bounding box) ve güven skorlarını yayınlar.⁶
- /yolo/tracking: Algılanan engellere benzersiz izleme kimlikleri (Tracking ID) atayarak ByteTrack veya BoT-SORT algoritmalarıyla hareket yönlerini takip eder.⁶
- /yolo/detections_3d: 2D algılama kutularını derinlik haritalarıyla eşleştirerek engellerin İDA'ya olan gerçek mesafesini ($3D$ koordinatlarını) üretir.⁶

Diğer taraftan, Glencsa/YOLOv8-ORB-SLAM3 deposu, otonom deniz araçlarının hem çevresel haritalama hem de dinamik engelleri filtreleme ihtiyaçlarını tek bir GPLv3 çatı altında birleştirmektedir.²⁶ Bu entegrasyon sayesinde, İDA eş zamanlı konumlandırma ve haritalama (SLAM) yaparken, YOLOv8 Nano vasıtasıyla hareketli engelleri haritadan dinamik olarak çıkarabilmektedir.²⁶ Alternatif olarak Warwick-Racing/yolov8_ros²⁷ ve BUPT-RobotTeam/bupt_rc_cv_inference_Yolov8TensorrtByteTrack²⁸ paketleri de doğrudan ROS 2 altında YOLOv8 Nano ve TensorRT kullanımına hazır GPLv3 çözümleri sunmaktadır.

Gömülü İşletim Sistemi Çekirdeklerinde YOLOv8 Nano Optimizasyonu

YOLOv8 Nano modelinin bir İDA üzerinde milisaniyeler seviyesinde gecikmeyle çalışabilmesi, kullanılan gömülü bilgisayarın işletim sistemi çekirdeğine (Kernel) uygun optimizasyon

yöntemlerinin uygulanmasını gerektirir. Modelin ham PyTorch (*yolov8n.pt*) formatında çalıştırılması işlemciyi aşırı yorarak otonom kontrol döngülerinde gecikmeye yol açacağından, Linux ve Android çekirdekleri için farklı donanım ivmelendirme mimarileri uygulanmalıdır.²¹

Linux Çekirdeği (NVIDIA Jetson / ARM64 Ubuntu) Optimizasyonu

NVIDIA Jetson Nano, Xavier NX veya Orin gibi Linux çekirdekli gömülü platformlarda YOLOv8 Nano modelini ivmelendirmenin en kararlı yöntemi, grafik yapısını doğrudan GPU çekirdeklerine göre derleyen **TensorRT** kütüphanesidir.²¹ Bu süreç, ardışık dönüştürme adımlarından oluşur:

1. **ONNX Modelleme:** Model, dinamik girdi desteği ve katman sadeleştirilmesi yapılarak ONNX formatına ihraç edilir²¹:

```
Python
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.export(format="onnx", simplify=True, imgsz=224)
```

2. **TensorRT Derleme (*trtexec*)**: Oluşturulan ONNX modeli, hedef gömülü cihazın Linux terminalinde TensorRT derleyicisi ile donanıma özel motor dosyasına dönüştürülür²¹:

Bash

```
trtexec tensorrt-8.5.1/texec-8.5.1/rtlib/rtlib.so -o /usr/local/lib/librtlib.so
```

Hassasiyet kaybı olmadan yüksek saniyedeki kare sayısı (FPS) elde etmek adına yarı duyarlıklı (FP16) kuantizasyon uygulanmalıdır.³³ Daha yüksek hızlar için, doğrulama veri kümesiyle ağırlık kalibrasyonu yapılarak sekiz bitlik tamsayı (INT8) kuantizasyonu da seçilebilmektedir.³³

3. **Uyumsuzluk Sorunlarının Çözümü**: Linux çekirdeğinde static/dynamic model derleme süreçlerinde veya JetPack sürümlerinde (örneğin TensorRT 8.5 ile 8.2 sürümleri arasında) oluşabilecek çakışmalar, MMDeploy ve OpenMMLab kütüphanelerinin kaynak koddan derlenmesiyle ve özel operatörlerin sisteme eklenmesiyle aşılabilmektedir.³³

AWS Greengrass V2 gibi uç cihaz yönetim servisleri aracılığıyla bu TensorRT motorları otonom olarak sahaya dağıtılabilen ve MQTT protokolü üzerinden tetiklenebilmektedir.²¹

Android Çekirdeği (Android NDK / NCNN / C++ Native) Optimizasyonu

Android çekirdeği üzerinde çalışan gömülü kartlarda (örneğin Rockchip veya Snapdragon tabanlı tek kart bilgisayarlar) CUDA çekirdekleri bulunmadığı için TensorRT kullanılamamaktadır. Bu platformlarda CPU ve Vulkan destekli mobil GPU'lardan maksimum verimi almak adına Tencent tarafından geliştirilen yüksek performanslı **NCNN** sinir ağı hesaplama çerçevesi tercih edilmelidir.²⁹

YOLOv8 Nano'nun Android çekirdeğinde NCNN ile optimize edilme adımları şunlardır²⁹:

1. **PNNX Entegrasyonu**: Model ihraç edilirken PyTorch Sinir Ağı Değişimi (PNNX) aracı aktif edilerek NCNN uyumlu grafik sadeleştirilmesi gerçekleştirilir.²⁹
2. **Hassas Çözünürlük Ölçeklemesi (*imgsz = 320*)**: Android'in gömülü işlemcilerindeki işlem yükünü azaltmak amacıyla girdi çözünürlüğü 320×320 piksele düşürülmelidir.²⁹ Bu sayede, algılama hassasiyeti kritik düzeyde düşmeden saniyedeki kare sayısı (FPS) katlanarak artar.²⁹
3. **C++ Native Katman Geliştirilmesi**: Android NDK (Native Development Kit) kullanılarak yazılan native-lib.cpp dosyası aracılığıyla, C++ düzeyinde bellek tahsisi gerçekleştirilir.²⁹ Bu optimizasyon, Java sanal makinesinin (JVM) çöp toplama (garbage collector) döngülerinden kaynaklanan mikro gecikmelerin önüne geçer.²⁹

Bununla birlikte, Android ekosisteminde çalışan başka bir GPLv3 otomasyon projesi olan steve1316/uma-android-automation, Android işletim sisteminde çalışan OpenCV ve Kotlin tabanlı sistemler için önemli bir optimizasyon yöntemi sunmaktadır.³⁵ Kameradan veya ekran yakalayıcıdan gelen girdilerin tutarlı bir şekilde YOLOv8 Nano tarafından işlenmesi için, Android pencere yöneticisi (WindowManager) ayarlarının Android Debug Bridge (ADB) komutları ile

sabitlenmesi gerekmektedir.³⁵

Görüntü analizinde boyut uyumsuzluğu yaşamamak adına kullanılan optimizasyon komutları şu şekildedir ³⁵:

Bash

Bu adımlar sayesinde, Android işletim sistemi arka planda çalışırken ekran çözünürlüğü ve inç başına düşen piksel yoğunluğu (DPI) yapay zeka modelinin eğitildiği standart girdi boyutuna zorlanarak algılama doğruluğu garanti altına alınır.³⁵ Ayrıca, bu arka plan servislerinin işletim sistemi tarafından sonlandırılmasını engellemek için Android Erişilebilirlik Servisi (Accessibility Service) ve ekran üzerinde çizim (Overlay) izinlerinin sisteme tanıtılması gerekmektedir.³⁵

Teknik Karşılaştırma ve Optimizasyon Matrisleri

Geliştirilen otonom İDA projesinin mimari kararlarını kolaylaştırmak amacıyla, GPLv3 lisanslı teknolojilerin Linux ve Android çekirdeklerindeki çalışma parametreleri ile kaynak tüketimleri aşağıdaki tablolarda yapılandırılmıştır.

Aşağıdaki tablo, YOLOv8 modelinin mgonzs13/yolo_ros sarmalayıcısı ile çalıştırıldığı durumdaki yaşam döngüsü düğümü (Lifecycle Node) durumuna göre kaynak tüketim farklarını göstermektedir:

Düğüm Durumu (Lifecycle State)	CPU Tüketimi (Intel i7 12. Nesil)	VRAM (Ekran Kartı Belleği)	Ağ Bant Genişliği (Bandwidth)	Referans
Aktif Durum (Active - 30 FPS)	%40 – (Tek Çekirdekte)	628 MB	En fazla 200 Mbps	⁶
Pasif Durum (Inactive)	%5 – (Tek Çekirdekte)	338 MB	0 - 20 Kbps	⁶

Bu veri, İDA'nın rihimde beklediği veya otonom seyrüseferde olmadığı durumlarda sistemin

pasif duruma çekilerek batarya ömrünün nasıl korunabileceğini göstermektedir.⁶
Aşağıdaki tabloda ise Linux ve Android işletim sistemi çekirdekleri üzerinde YOLOv8 Nano modelinin optimizasyon süreçleri karşılaştırılmıştır:

Optimizasyon Parametresi	Linux Çekirdeği (Ubuntu / ROS 2)	Android Çekirdeği (Android NDK)
Öncelikli Araç Seti	TensorRT / ONNX Runtime ²¹	NCNN / PNNX ²⁹
İvmelendirici Donanım Birimi	NVIDIA CUDA / Tensor Çekirdekleri ²¹	Mobil ARM CPU / Vulkan GPU API ²⁹
Model Girdi Çözünürlüğü (<i>imgsz</i>)	640 × veya 640 × ²¹	320 × 320 (Performans Odaklı) ²⁹
Desteklenen Veri Hassasiyeti	FP16 veya INT8 (Kalibrasyon Dosyası ile) ³³	FP16 (Dahili Sadeleştirilmiş Katmanlar) ²⁹
Sistem İzin Gereksinimleri	Standart Kullanıcı / Docker Ayrıcalıkları ²¹	Erişilebilirlik Servisi ve Ekran Üstü Çizim İzni ³⁵
Görüntü Önışlem Kütüphanesi	C++ OpenCV / ROS Image Transport ⁶	C++ NDK OpenCV / Android Bitmap ²⁹

Entegrasyon ve Mimari Yol Haritası Önerileri

Yapılan araştırmalar ve elde edilen hazır teknolojiler doğrultusunda, tamamen GPLv3 lisans uyumlu, Linux veya Android üzerinde koşan otonom bir İDA platformunun sistem mimarisi şu şekilde kurgulanmalıdır:

- **Sistem Kontrol Mimarı (Alt Seviye):** İDA'nın dümen ve motor kontrolünde kesinlikle **ArduPilot (Rover/Boat)** yazılımı kullanılmalıdır.⁹ Rüzgarlı koşullarda dalga salınımını engellemek için konum kontrolörü kazancı ($PSC_VEL_P > 3$) yüksek tutulmalı ve pivot dönüşler engellenerek pruva savrulması minimize edilmelidir.¹⁶
- **Görsel Algılama Algoritması (Üst Seviye):** YOLOv8 Nano modeli boş COCO ağırlıklarıyla değil, kesinlikle **MaSTr1325** ve **MODD2** veri kümeleri kullanılarak deniz engelleri için özel olarak eğitilmelidir.²² Eğitilen bu model, su yüzeyini algılayan bir semantik bölütleme ağı ile maskelenerek yanlış alarmlardan temizlenmelidir.²²
- **Platform Seçimi ve Optimizasyon:**
 - **Eğer Linux (Jetson) Tercih Edilirse:** ROS 2 Humble işletim sistemi kurulmalı, mgonzs13/yolo_ros paketi entegre edilmeli ve YOLOv8 Nano modeli TensorRT (.engine)

FP16 formatında çalıştırılmalıdır.⁶

- **Eğer Android (ARM) Tercih Edilirse:** Kotlin/C++ tabanlı native NDK mimarisi kurgulanmalı, model PNNX ile NCNN formatına sadeleştirilerek ihraç edilmeli ve ekran çözünürlüğü ADB üzerinden sabitlenerek 320×320 girdi çözünürlüğünde çalıştırılmalıdır.²⁹
- **Yasal Lisans Uyumluluğu:** Kullanılan tüm temel kütüphanelerin (YOLOv8, ArduPilot, yolo_ros) GPLv3 lisans yapısına sahip olması nedeniyle, bu sistemler üzerine inşa edilen otonom İDA seyrüsefer yazılımlarının da açık kaynak kodlu olarak yayınlanması yasal bir zorunluluktur.⁹

Alıntılanan çalışmalar

1. ida-plugin.json - JusticeRage/Gepetto - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/JusticeRage/Gepetto/blob/main/ida-plugin.json>
2. GitHub - EliseZeroTwo/IDA-NDS: IDAPython Loader for DS and DSi ROMs, erişim tarihi Mayıs 21, 2026, <https://github.com/EliseZeroTwo/IDA-NDS>
3. idapm is IDA Plugin Manager via GitHub Repository., erişim tarihi Mayıs 21, 2026, <https://github.com/tkmru/idapm>
4. ghassani/mclf-ida-loader: An IDA file loader for Mobicore trustlet and driver binaries - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/ghassani/mclf-ida-loader>
5. REhints/efiXplorer: IDA plugin and loader for UEFI firmware analysis and reverse engineering automation - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/binarly-io/efixplorer>
6. mgonzs13/yolo_ros: Ultralytics YOLOv8, YOLOv9 ... - GitHub, erişim tarihi Mayıs 21, 2026, https://github.com/mgonzs13/yolo_ros
7. Simulation assets for the rowbot team - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/USYD-RowBot/simulation>
8. Perdixky/ROS2-USV - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/Perdixky/ROS2-USV>
9. ardupilot_wiki/dev/source/docs/license-gplv3.rst at master - GitHub, erişim tarihi Mayıs 21, 2026, https://github.com/ArduPilot/ardupilot_wiki/blob/master/dev/source/docs/license-gplv3.rst
10. README.md - Perdixky/ROS2-USV - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/Perdixky/ROS2-USV/blob/master/README.md>
11. ASM Studios - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/ASM-Studios>
12. Marnonel6/ROS2_offboard_drone_control: ROS2 Off-board drone control with a companion computer and PX4 over DDS (serial). Autonomous drone with path planning. - GitHub, erişim tarihi Mayıs 21, 2026, https://github.com/Marnonel6/ROS2_offboard_drone_control
13. Histo-Planner : A Real-time Local Planner for MAVs Teleoperation based on Histogram of Obstacle Distribution - GitHub, erişim tarihi Mayıs 21, 2026,

- <https://github.com/w407022008/histo-planner>
14. GitHub - ArduPilot/ardupilot: ArduPlane, ArduCopter, ArduRover, ArduSub source, erişim tarihi Mayıs 21, 2026, <https://github.com/ardupilot/ardupilot>
 15. ardupilot_wiki/common/source/docs/common-history-of-ardupilot.rst at master - GitHub, erişim tarihi Mayıs 21, 2026, https://github.com/ArduPilot/ardupilot_wiki/blob/master/common/source/docs/common-history-of-ardupilot.rst
 16. yuri-rage/yard-pilot: Path planning companion for ArduPilot's Mission Planner - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/yuri-rage/yard-pilot>
 17. SMaRC - Swedish Maritime Robotics Centre · GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/smarc-project>
 18. YOLOv8 in PyTorch > ONNX > CoreML > TFLite - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/augmentedstartups/yolov8>
 19. autogyro/yolo-V8: YOLOv8 in PyTorch > ONNX > CoreML > TFLite - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/autogyro/yolo-V8>
 20. YOLOv8 Face in PyTorch > ONNX > CoreML > TFLite - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/jags111/yolov8-face>
 21. deploy-yolov8-on-edge-using-aws-iot-greengrass/README.md at main - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/aws-samples/deploy-yolov8-on-edge-using-aws-iot-greengrass/blob/main/README.md>
 22. Timotej979/U-Net-Water-Land-Segmentation: Water/land ... - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/Timotej979/U-Net-Water-Land-Segmentation>
 23. bborja/modd: Dataset and Evaluation Scripts for Obstacle Detection via Semantic Segmentation in a Marine Environment - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/bborja/modd>
 24. HyAgOsK/Traffic-Tracker - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/HyAgOsK/Traffic-Tracker>
 25. NEW - YOLOv8 Multi-Object Tracking · Issue #1429 - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/ultralytics/ultralytics/issues/1429>
 26. YOLOv8-ORB-SLAM3: Semantic SLAM with dynamic feature point removal · GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/Glencsa/YOLOv8-ORB-SLAM3>
 27. Warwick-Racing/yolov8_ros: Ultralytics YOLOv8 and ... - GitHub, erişim tarihi Mayıs 21, 2026, https://github.com/Warwick-Racing/yolov8_ros
 28. BUPT-RobotTeam - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/BUPT-RobotTeam>
 29. mpj1234/ncnn-yolov13-android: Yolov13 model supports ... - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/mpj1234/ncnn-yolov13-android>
 30. aws-samples/deploy-yolov8-on-edge-using-aws-iot-greengrass - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/aws-samples/deploy-yolov8-on-edge-using-aws-iot-greengrass>
 31. NTU-ICG/multidrone-detection-tracking: A multi-drone detection and tracking system using YOLOv5, TensorRT, Deep SORT and ROS deployed on Jetson Xavier

- NX - GitHub, erişim tarihi Mayıs 21, 2026,
<https://github.com/NTU-ICG/multidrone-detection-tracking>
32. kuanshi1718/CFSLFE - GitHub, erişim tarihi Mayıs 21, 2026,
<https://github.com/kuanshi1718/CFSLFE>
33. sktedro/vehicle_detection_for_embedded_platforms: Vehicle detection and tracking in a surveillance type of a video using a convolutional neural network - GitHub, erişim tarihi Mayıs 21, 2026,
https://github.com/sktedro/vehicle_detection_for_embedded_platforms
34. onuralpszr/GFPGAN-ncnn-vulkan: [WIP] NCNN with Vulkan implementation of GFPGAN aims at developing Practical Algorithms for Real-world Face Restoration · GitHub, erişim tarihi Mayıs 21, 2026,
<https://github.com/onuralpszr/GFPGAN-ncnn-vulkan>
35. GitHub - steve1316/uma-android-automation: Android application written in Kotlin with the sole purpose of completely automating a run of Uma Musume Pretty Derby., erişim tarihi Mayıs 21, 2026,
<https://github.com/steve1316/uma-android-automation>
36. GPL 3.0 license comments #5710 - ultralytics/yolov5 - GitHub, erişim tarihi Mayıs 21, 2026, <https://github.com/ultralytics/yolov5/issues/5710>